Лабораторная работа №5

Вероятностные алгоритмы проверки чисел на простоту

Кадрова Ангелина Александровна

Содержание

1 Введение **4**
1.1 Цели и задачи 4
1.2 Теоретическое введение 4
1.2.1 Тест Ферма 5
1.2.2 Тест Соловэя-Штрассена 5
1.2.3 Тест Миллера-Рабина 5

2 Выполнение лабораторной работы **6**
2.1 Тест Ферма 6
2.2 Алгоритм вычисления символа Якоби 6
2.3 Тест Соловэя-Штрассена 8
2.4 Тест Миллера-Рабина 9
2.5 Тестирование алгоритмов 10

3 Вывод **12**

Список литературы **13**

Список иллюстраций

2.1 Результаты работы алгоритмов 11
2.2 Результаты работы алгоритмов 11

1 Введение

1.1 Цели и задачи

Цель работы

Основная цель работы — изучить и реализовать вероятностные алгоритмы проверки чисел на простоту.

Задание

С помощью языка программирования Julia реализовать алгоритмы:

1. Тест Ферма;
2. Алгоритм вычисления символа Якоби;
3. Тест Соловэя-Штрассена;
4. Тест Миллера-Рабина.

1.2 Теоретическое введение

Целое число $p \in \mathbb{Z} \setminus \{0\}$ называется *простым*, если оно не является делителем единицы и не имеет других делителей, кроме тривиальных ($\pm 1, \pm p$). В противном случае число $p \in \mathbb{Z} \setminus \{-1, 0, 1\}$ называется *составным*[1].

Целые числа a и b называются *сравнимыми по модулю* m ($a \equiv b \pmod{m}$), если разность $a - b$ делится на m.

Алгоритмы проверки на простоту делятся на *детерминированные* (гарантированно решают задачу) и *вероятностные* (используют генератор случайных чисел и дают не гарантированно точный ответ).

Для проверки числа n вероятностным алгоритмом выбирают случайное число a ($1 < a < n$) и проверяют условия теста. Если n не проходит тест — оно составное. Если проходит — вероятно простое. После t независимых выполнений вероятность ошибки не превосходит $\frac{1}{2^t}$.

1.2.1 Тест Ферма

Основан на малой теореме Ферма: для простого p и $1 \leq a \leq p-1$ выполняется:

$$a^{p-1} \equiv 1 \pmod{p}$$

Если для нечётного n существует a, что НОД$(a, n) = 1$ и $a^{n-1} \not\equiv 1 \pmod{n}$, то n составное.

1.2.2 Тест Соловэя-Штрассена

Основан на критерии Эйлера: нечётное n простое тогда и только тогда, когда для любого a, взаимно простого с n:

$$a^{\frac{n-1}{2}} \equiv \left(\frac{a}{n}\right) \pmod{n}$$

где $\left(\frac{a}{n}\right)$ — символ Якоби. Для $n = p_1 p_2 \ldots p_r$:

$$\left(\frac{m}{n}\right) = \left(\frac{m}{p_1}\right)\left(\frac{m}{p_2}\right)\ldots\left(\frac{m}{p_r}\right)$$

1.2.3 Тест Миллера-Рабина

Пусть n нечётное и $n - 1 = 2^s r$, где r нечётное. Если n простое, то для любого $a \geq 2$, взаимно простого с n, выполняется $a^{n-1} \equiv 1 \pmod{n}$. Тест проверяет дополнительные условия для повышения надёжности[1].

Julia — высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений[2].

2 Выполнение лабораторной работы

2.1 Тест Ферма

```
function ferma_test(n)
    if n < 5
        error("Число должно быть >= 5")
    end
    a = rand(2:(n-2))
    r = powermod(a, n-1, n)
    if r == 1
        println("Число $n, вероятно, простое\n")
        return true
    else
        println("Число $n составное\n")
        return false
    end
end
```

2.2 Алгоритм вычисления символа Якоби

```
function jacobi_symbol(a, n)
    if n < 3 || n % 2 == 0
        error("n должно быть нечетным и >= 3")
```

```
end
if a < 0 || a >= n
    error("a должно быть в диапозоне [0,n)")
end
g = 1
while true
    if a == 0
        return 0
    end
    if a == 1
        return g
    end
    k = 0
    a1 = a
    while a1 % 2 == 0
        a1 = a1 ÷ 2
        k = k + 1
    end
    if k % 2 == 0
        s = 1
    else
        n_mod_8 = n % 8
        if n_mod_8 == 1 || n_mod_8 == 7
            s = 1
        else
            s = -1
        end
    end
    if a1 == 1
```

```
            return g * s
        end
        if n % 4 == 3 && a1 % 4 == 3
            s = -s
        end
        a = n % a1
        n = a1
        g = g * s
    end
end
```

2.3 Тест Соловэя-Штрассена

```
function solovay_strassen_test(n)
    if n < 5
        error("Число должно быть >= 5")
    end
    a = rand(2:(n-2))
    exponent = (n - 1) ÷ 2
    r = powermod(a, exponent, n)
    if r != 1 && r != n - 1
        println("Число $n составное\n")
        return false
    end
    j = jacobi_symbol(a, n)
    if r == (j % n + n) % n
        println("Число $n, вероятно, простое\n")
        return true
    else
```

```
        println("Число $n составное\n")
        return false
    end
end
```

2.4 Тест Миллера-Рабина

```
function miller_rabin_test(n)
    if n < 5
        error("Число должно быть >= 5")
    end
    r = n - 1
    s = 0
    while r % 2 == 0
        r = r ÷ 2
        s = s + 1
    end
    a = rand(2:(n-2))
    y = powermod(a, r, n)
    if y != 1 && y != n -1
        j = 1
        while j <= s - 1
            if y != n - 1
                y = powermod(y, 2, n)
                if y == 1
                    println("Число $n составное\n")
                    return false
                end
            end
```

```
            j = j + 1
        end
        if y != n - 1
            println("Число $n составное\n")
            return false
        end
    end
    println("Число $n, вероятно, простое\n")
    return true
end
```

2.5 Тестирование алгоритмов

```
test_numbers_prime = [17, 29, 97]
for n in test_numbers_prime
    println("\nПроверяем число: $n (простое)")
    ferma_test(n)
    solovay_strassen_test(n)
    miller_rabin_test(n)
end

test_numbers_composite = [33, 91, 121]
for n in test_numbers_composite
    println("\nПроверяем число: $n (составное)")
    ferma_test(n)
    solovay_strassen_test(n)
    miller_rabin_test(n)
end
```

В результате получим следующий вывод(рис. 2.1-2.2):

```
test_numbers_prime = [17, 29, 97]
for n in test_numbers_prime
    println("\nПроверяем число: $n (простое)")
    ferma_test(n)
    solovay_strassen_test(n)
    miller_rabin_test(n)
end
```

✓ 0.0s

```
Проверяем число: 17 (простое)
Число 17, вероятно, простое

Число 17, вероятно, простое

Число 17, вероятно, простое


Проверяем число: 29 (простое)
Число 29, вероятно, простое

Число 29, вероятно, простое

Число 29, вероятно, простое


Проверяем число: 97 (простое)
Число 97, вероятно, простое

Число 97, вероятно, простое

Число 97, вероятно, простое
```

Рис. 2.1: Результаты работы алгоритмов

```
test_numbers_composite = [33, 91, 121]
for n in test_numbers_composite
    println("\nПроверяем число: $n (составное)")
    ferma_test(n)
    solovay_strassen_test(n)
    miller_rabin_test(n)
end
```

✓ 0.1s

```
Проверяем число: 33 (составное)
Число 33 составное

Число 33 составное

Число 33 составное


Проверяем число: 91 (составное)
Число 91 составное

Число 91 составное

Число 91 составное


Проверяем число: 121 (составное)
Число 121 составное

Число 121 составное

Число 121 составное
```

Рис. 2.2: Результаты работы алгоритмов

3 Вывод

С помощью языка программирования Julia были реализованы вероятностные алгоритмы проверки чисел на простоту.

Список литературы

1. Математические основы защиты информации и информационной безопасности. Лабораторная работа №5: Вероятностные алгоритмы проверки чисел на простоту: Методические указания к лабораторной работе. Саратовский государственный университет имени Н.Г. Чернышевского, 2026.
2. JuliaLang [Электронный ресурс]. 2026 JuliaLang.org contributors. URL: https://julialang.org/ (дата обращения: 27.10.2026).